

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

**Кафедра автоматизированных систем управления
(АСУ)**

РАЗРАБОТКА ВЕБ-СЕРВИСА

Лабораторная работа №2 по дисциплине «Разработка веб-сервисов для
научных и прикладных задач» (Вариант 10)

Выполнила:
Студентка гр. 3-433П8-5

Медведева Юлия Евгеньевна

«28» августа 2026г.
(дата)

Проверил:

(должность, ученая степень, звание)
/ _____ /
(подпись) И. О. Фамилия
« ____ » _____ 20 ____ г.
(дата)

СОДЕРЖАНИЕ

Введение	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1 АРХИТЕКТУРНЫЙ СТИЛЬ REST	5
1.2 FLASK И FLASGGER.....	5
1.3 HTTP-МЕТОДЫ, КОДЫ СОСТОЯНИЯ И ФОРМАТ JSON	6
1.4 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ И СТРУКТУРА ПРОЕКТА	6
2 ОПИСАНИЕ ХОДА ВЫПОЛНЕНИЯ РАБОТЫ.....	8
Заключение.....	14
Список литературы.....	15
Приложение А.....	16
А.1 Файл app.py	16

Введение

Цель работы — углубить понимание принципов проектирования веб-сервисов, освоив разработку REST API на языке Python с использованием микрофреймворка Flask. Практической задачей является создание программного интерфейса для работы с коллекцией музыкальных произведений, поддерживающего основные операции управления ресурсами, сортировку, агрегатные вычисления, валидацию входных данных и автоматическое формирование документации API.

Основные задачи:

1. Спроектировать структуру ресурса «Музыкальное произведение», включающего название, исполнителя, жанр, длительность, год выпуска и рейтинг.
2. Реализовать REST API на Flask для получения списка ресурсов и отдельного ресурса по идентификатору.
3. Реализовать операции создания, обновления и удаления ресурсов (CRUD) с корректной обработкой ошибок.
4. Реализовать сортировку коллекции по выбранному полю в прямом и обратном порядке, а также вычисление среднего, максимального и минимального значений для числовых полей.
5. Реализовать валидацию входных данных и передачу данных в формате JSON с использованием соответствующих HTTP-кодов состояния.
6. Настроить автоматическое формирование интерактивной документации Swagger/OpenAPI с использованием расширения Flasgger.

REST (Representational State Transfer) определяет принципы взаимодействия клиента и сервера через унифицированный интерфейс и стандартные HTTP-методы. В рамках лабораторной работы ресурсы представлены музыкальными произведениями, каждое из которых имеет собственный идентификатор и набор характеристик. При использовании Flask можно реализовать маршрутизацию запросов и обработку JSON-

данных, а с помощью Flasgger — сформировать интерактивную документацию API на основе описаний маршрутов.

Корректное применение HTTP-методов и кодов состояния делает поведение веб-сервиса предсказуемым для клиента. Валидация входных данных предотвращает сохранение некорректных значений, а сортировка и агрегатные операции расширяют возможности получения и анализа информации через API.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В данной лабораторной работе применяются основные принципы построения REST API: ресурсная модель, использование HTTP-методов, обмен данными в формате JSON, обработка кодов состояния и автоматическое документирование интерфейса. Ниже представлено их теоретическое обоснование.

1.1 АРХИТЕКТУРНЫЙ СТИЛЬ REST

REST (Representational State Transfer) — архитектурный стиль построения веб-сервисов, в котором сущности предметной области рассматриваются как ресурсы и адресуются с помощью URL. Операции над ресурсами выполняются стандартными HTTP-методами: GET используется для чтения данных, POST — для создания ресурса, PUT или PATCH — для обновления, DELETE — для удаления.

В работе ресурсом является музыкальное произведение. Коллекция ресурсов доступна по адресу `/tracks`, отдельный объект — по адресу `/tracks/<id>`, а агрегатные характеристики коллекции — по адресу `/tracks/stats`.

1.2 FLASK И FLASGGER

Flask — микрофреймворк для разработки веб-приложений на Python. Он предоставляет механизм маршрутизации HTTP-запросов, обработку входных параметров и формирование ответов, в том числе в формате JSON. Благодаря минимальному набору встроенных компонентов Flask подходит для создания небольших REST-сервисов и позволяет явно контролировать структуру приложения.

Flasgger — расширение Flask, предназначенное для автоматического формирования документации Swagger UI / OpenAPI. В разработанном приложении описание маршрутов задаётся в YAML-блоках docstring-функций, после чего интерактивная документация становится доступна по адресу `/apidocs/`.

1.3 HTTP-МЕТОДЫ, КОДЫ СОСТОЯНИЯ И ФОРМАТ JSON

Для обмена данными между клиентом и сервером используется формат JSON. Сервер принимает JSON-объекты при создании и обновлении ресурсов и возвращает JSON-представление данных в ответах.

Результат выполнения запроса дополнительно характеризуется HTTP-кодом состояния. В работе используются 200 OK для успешного чтения или обновления, 201 Created для успешного создания ресурса, 204 No Content для успешного удаления без тела ответа, 400 Bad Request при ошибках валидации и 404 Not Found при обращении к отсутствующему ресурсу.

1.4 ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ И СТРУКТУРА ПРОЕКТА

Приложение реализовано на языке Python 3.10 с использованием фреймворка Flask и расширения Flasgger. Для запуска приложения также используется WSGI-сервер Gunicorn.

Используемые библиотеки: Flask 3.0.3, Flasgger 0.9.7.1 и Gunicorn 22.0.0.

Проект содержит основной модуль `app.py` с маршрутами, валидацией и Swagger-описаниями, файл `requirements.txt` с зависимостями и `.gitignore` с перечнем исключаемых из репозитория файлов. Коллекция `tracks` хранится в памяти процесса в виде списка словарей без использования внешней базы данных. Идентификаторы ресурсов назначаются последовательно с помощью счётчика `next_id`.

Поля ресурса: id — идентификатор, назначаемый сервером; title — название произведения; artist — исполнитель; genre — жанр; duration_sec — длительность в секундах; year — год выпуска; rating — рейтинг.

2 ОПИСАНИЕ ХОДА ВЫПОЛНЕНИЯ РАБОТЫ

1. Создан проект Flask: app.py, виртуальное окружение venv, файл зависимостей requirements.txt.
2. Определена структура ресурса «музыкальное произведение» и стартовый набор данных (3 записи) для демонстрации.
3. Реализован маршрут GET /tracks — список всех произведений с опциональной сортировкой по параметрам sort_by (любое поле из ALL_FIELDS) и order (asc/desc, по умолчанию asc).
4. Реализован маршрут GET /tracks/stats — агрегаты (avg, max, min) по числовым полям duration_sec, year, rating.
5. Реализован маршрут GET /tracks/<id> — получение одного произведения; при отсутствии id возвращается 404 с телом {"error": "track not found"}.
6. Реализован маршрут POST /tracks — создание записи с полной валидацией (validate_payload, partial=False): все поля обязательны, строковые проверяются на тип str, числовые — на тип int/float; при ошибках — 400 со списком сообщений.
7. Реализован маршрут PUT /tracks/<id> — частичное обновление (partial=True, обязательности полей нет, но типы проверяются); 404, если id не найден.
8. Реализован маршрут DELETE /tracks/<id> — удаление записи, ответ 204 без тела; 404, если id не найден.
9. Каждый маршрут снабжён YAML-докстрингом для Flasgger — документация доступна автоматически по адресу /apidocs/ без дополнительного описания схемы.
10. Приложение проверено локально запросами через curl (см. раздел 6).

После реализации основных маршрутов была выполнена проверка работы REST API на тестовом наборе данных. Запрос GET /tracks возвращает список музыкальных произведений. При передаче параметров

sort_by=year и order=desc коллекция сортируется по году выпуска в порядке убывания. Ниже приведены скриншоты запросов и ответов для каждого метода (рисунки).

Запрос GET /tracks без параметров возвращает список всех произведений (рисунок 1).

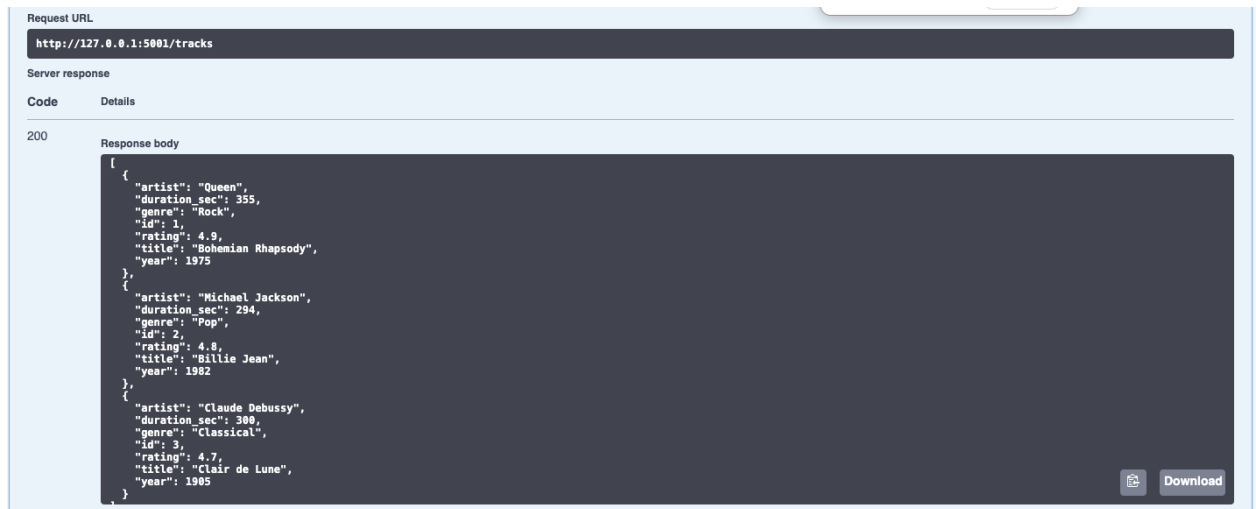


Рисунок 1 — GET /tracks (список произведений без сортировки)

При передаче параметров sort_by=year и order=desc та же коллекция сортируется по году выпуска в порядке убывания (рисунок 2).



Рисунок 2 GET /tracks?sort_by=year&order=desc — (сортировка по году убывания)

Маршрут GET /tracks/stats возвращает агрегаты (avg, max, min) по числовым полям duration_sec, year, rating (рисунок 3).



Рисунок 3 — GET /tracks/stats (агрегаты по числовым полям)

Запрос POST /tracks с телом {"title": "Blue in Green", "artist": "Bill Evans Trio", "genre": "Modal Jazz", "duration_sec": 337, "year": 1959, "rating": 4.9} (рисунок 4) создаёт новую запись — ответ 201 с присвоенным id: 4 (рисунок 5).

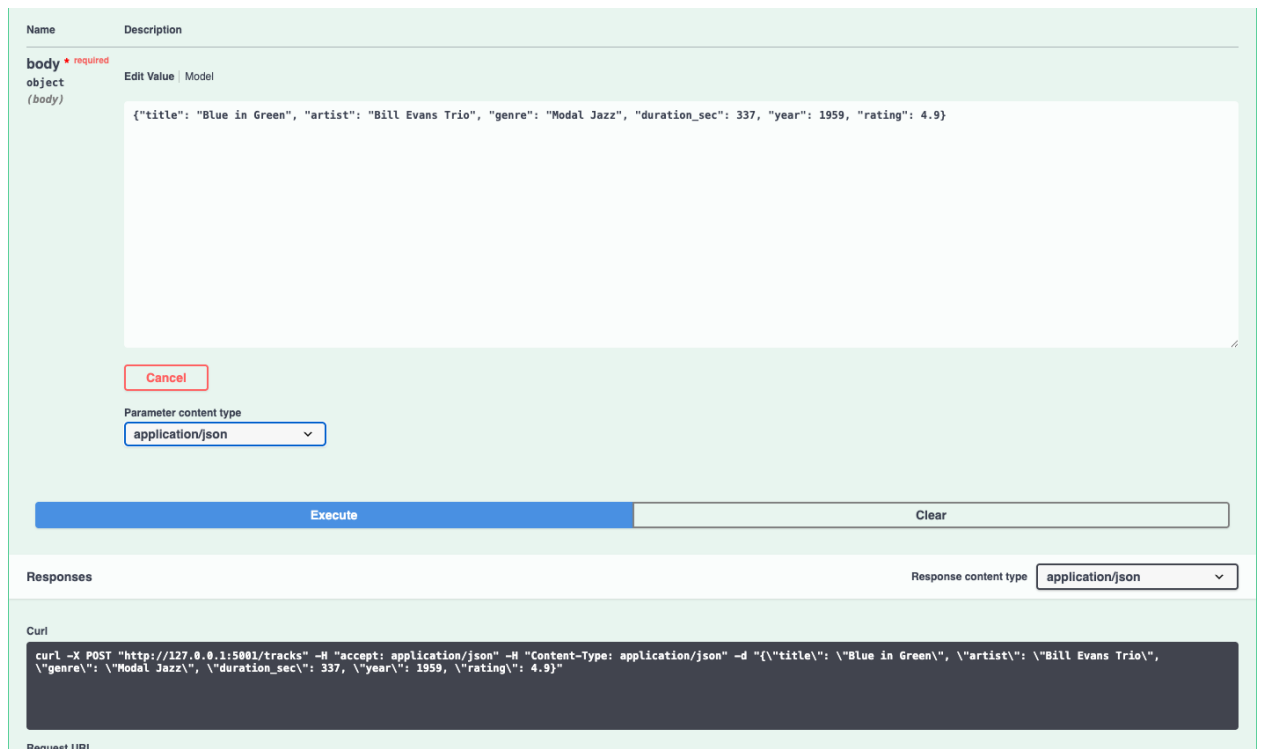


Рисунок 4 — POST /tracks тело запроса

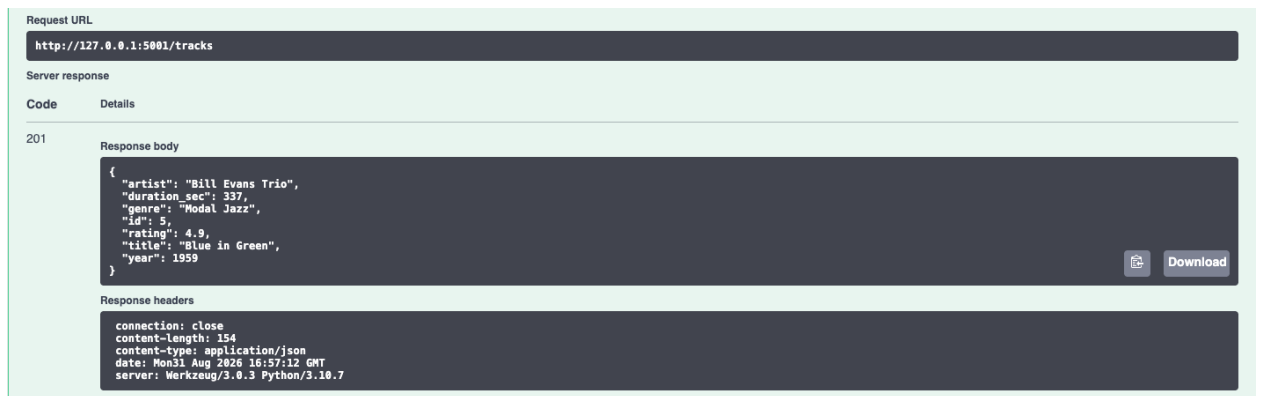


Рисунок 5 — POST /tracks (ответ на создание записи)

Запрос PUT /tracks/1 с телом {"rating": 5.0} обновляет только поле rating, остальные поля записи сохраняются — ответ 200 (рисунок 6).

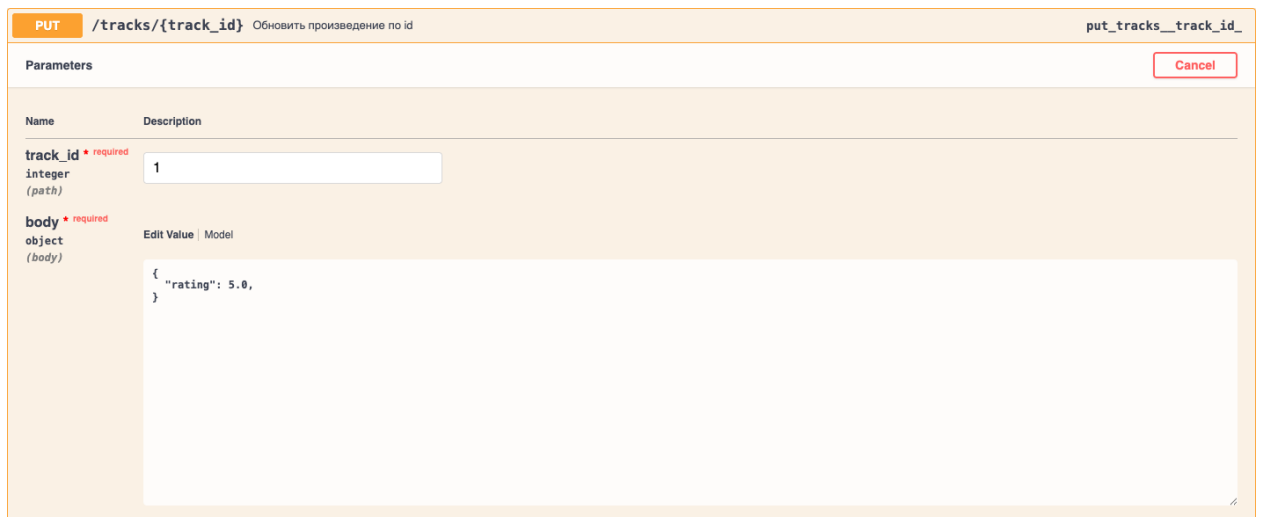


Рисунок 6 — PUT /tracks/1 (запрос на частичное обновление)

Запрос DELETE /tracks/5 удаляет запись — ответ 204 без тела (рисунок 6).



Рисунок 7 — DELETE /tracks/5

Запрос GET /tracks/999 к несуществующему id возвращает 404 с телом {"error": "track not found"} (рисунок 8).



Рисунок 8 — GET /tracks/999 (несуществующий id, 404)

Запрос POST /tracks с неполным телом `{"title":"x"}` возвращает 400 со списком ошибок валидации: `artist is required`, `genre is required`, `duration_sec is required`, `year is required`, `rating is required` (рисунок 9).

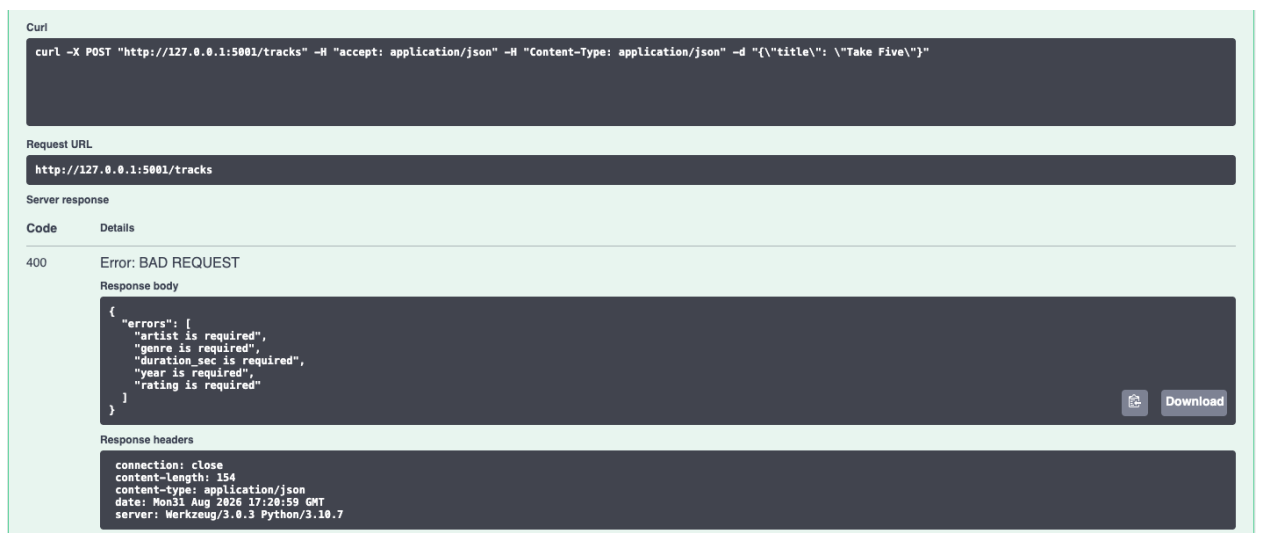


Рисунок 9 — POST /tracks с неполным телом (400, ошибки валидации)

Проверка показала, что приложение корректно обрабатывает успешные запросы, отсутствие ресурсов и ошибки входных данных. Для маршрутов сформирована интерактивная документация Swagger UI, доступная по адресу `/apidocs/` (рисунок 10).

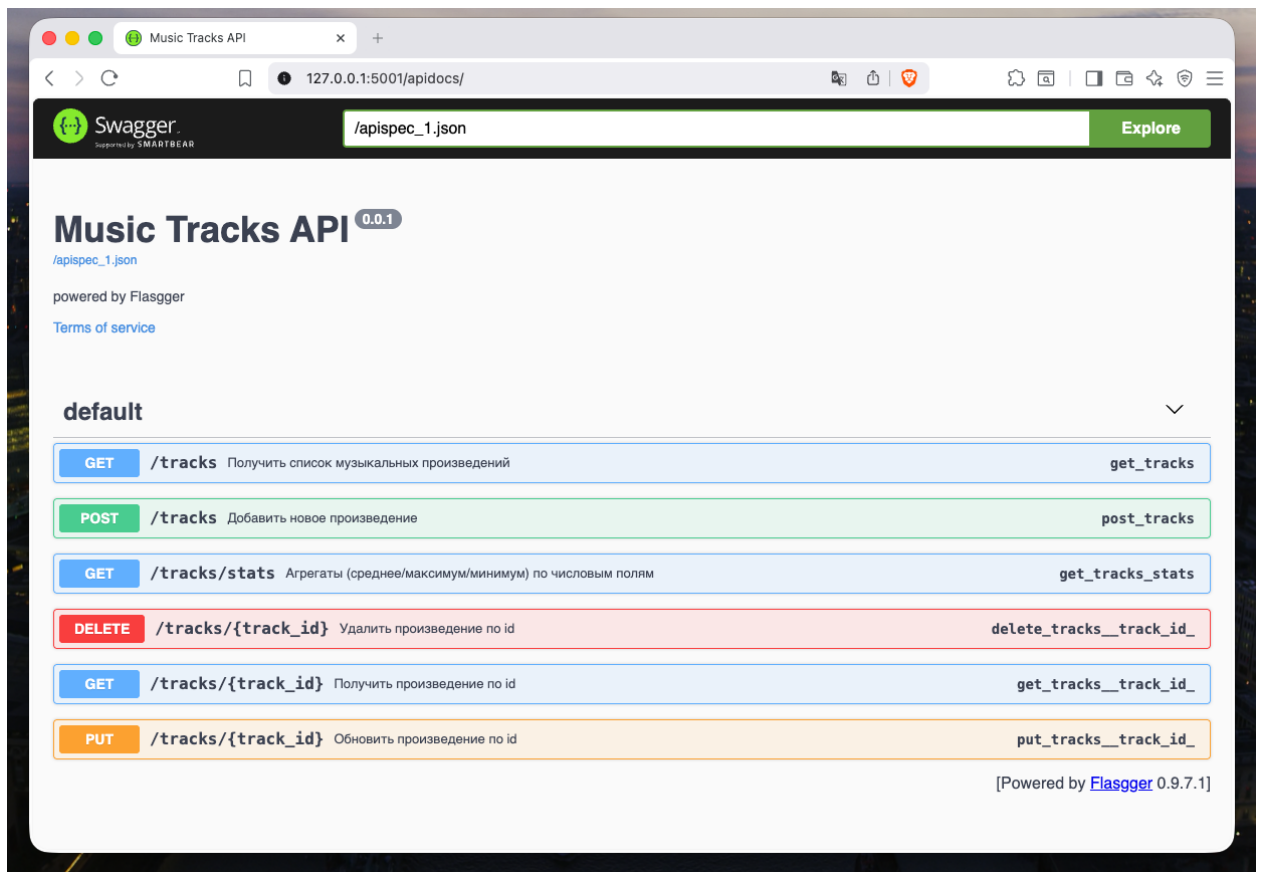


Рисунок 10 — Swagger UI (/apidocs/) со списком эндпоинтов

Заключение

В ходе выполнения лабораторной работы были изучены и применены на практике основные принципы проектирования REST API на языке Python с использованием Flask. Для предметной области «Музыкальные произведения» разработан программный интерфейс, обеспечивающий получение коллекции и отдельных ресурсов, создание, обновление и удаление записей.

Реализована сортировка списка по выбранному полю в прямом и обратном порядке, а также вычисление среднего, максимального и минимального значений для числовых характеристик. Для повышения надёжности API добавлена валидация входных данных и обработка ошибочных ситуаций с использованием корректных HTTP-кодов состояния 200, 201, 204, 400 и 404.

Документирование интерфейса выполнено с помощью расширения Flasgger: Swagger UI формируется автоматически на основе YAML-описаний маршрутов и позволяет просматривать доступные операции API. В результате выполнения работы были закреплены навыки построения ресурсно-ориентированных веб-сервисов, организации обмена JSON-данными, обработки HTTP-запросов и документирования программного интерфейса. Поставленные цели лабораторной работы достигнуты.

Список литературы

1. Разработка веб-сервисов для научных и прикладных задач: методические указания по выполнению лабораторных работ. — Томск: ТУСУР.
2. Flask Documentation. — URL: <https://flask.palletsprojects.com/>
3. Flasgger Documentation. — URL: <https://github.com/flasgger/flasgger>
4. Fielding, R. Architectural Styles and the Design of Network-based Software Architectures (REST). — 2000.

Приложение А

Файлы проекта

А.1 Файл app.py

```
from flask import Flask, jsonify, request
from flasgger import Swagger

app = Flask(__name__)
app.config["SWAGGER"] = {
    "title": "Music Tracks API",
    "uiversion": 3,
}
swagger = Swagger(app)

STRING_FIELDS = ("title", "artist", "genre")
NUMERIC_FIELDS = ("duration_sec", "year", "rating")
ALL_FIELDS = STRING_FIELDS + NUMERIC_FIELDS

tracks = [
    {"id": 1, "title": "Bohemian Rhapsody", "artist": "Queen", "genre": "Rock",
     "duration_sec": 355, "year": 1975, "rating": 4.9},
    {"id": 2, "title": "Billie Jean", "artist": "Michael Jackson", "genre": "Pop",
     "duration_sec": 294, "year": 1982, "rating": 4.8},
    {"id": 3, "title": "Clair de Lune", "artist": "Claude Debussy", "genre":
     "Classical",
     "duration_sec": 300, "year": 1905, "rating": 4.7},
]
next_id = 4

def find_track(track_id):
    return next((t for t in tracks if t["id"] == track_id), None)

def validate_payload(data, partial=False):
    errors = []
    for f in STRING_FIELDS:
        if f in data and not isinstance(data[f], str):
            errors.append(f' {f} must be a string')
        if not partial and f not in data:
            errors.append(f' {f} is required')
    for f in NUMERIC_FIELDS:
        if f in data and not isinstance(data[f], (int, float)):
```

```

        errors.append(f" {f} must be a number")
    if not partial and f not in data:
        errors.append(f" {f} is required")
    return errors

```

```

@app.route("/tracks", methods=["GET"])
def list_tracks():
    """
    Получить список музыкальных произведений
    ---
    parameters:
      - name: sort_by
        in: query
        type: string
        required: false
        enum: [title, artist, genre, duration_sec, year, rating]
        description: Поле для сортировки
      - name: order
        in: query
        type: string
        required: false
        enum: [asc, desc]
        description: Направление сортировки (по умолчанию asc)
    responses:
      200:
        description: Список произведений
    """
    sort_by = request.args.get("sort_by")
    order = request.args.get("order", "asc")
    result = list(tracks)
    if sort_by in ALL_FIELDS:
        result.sort(key=lambda t: t[sort_by], reverse=(order == "desc"))
    return jsonify(result)

```

```

@app.route("/tracks/stats", methods=["GET"])
def stats():
    """
    Агрегаты (среднее/максимум/минимум) по числовым полям
    ---
    responses:
      200:
        description: Статистика по duration_sec, year, rating
    """

```

```

if not tracks:
    return jsonify({f: None for f in NUMERIC_FIELDS})
result = {}
for f in NUMERIC_FIELDS:
    values = [t[f] for t in tracks]
    result[f] = {
        "avg": sum(values) / len(values),
        "max": max(values),
        "min": min(values),
    }
return jsonify(result)

```

```

@app.route("/tracks/<int:track_id>", methods=["GET"])
def get_track(track_id):
    """
    Получить произведение по id
    ---
    parameters:
      - name: track_id
        in: path
        type: integer
        required: true
    responses:
      200:
        description: Найденное произведение
      404:
        description: Произведение не найдено
    """
    track = find_track(track_id)
    if track is None:
        return jsonify({"error": "track not found"}), 404
    return jsonify(track)

```

```

@app.route("/tracks", methods=["POST"])
def create_track():
    """
    Добавить новое произведение
    ---
    parameters:
      - name: body
        in: body
        required: true
        schema:

```

```

    type: object
    required: [title, artist, genre, duration_sec, year, rating]
    properties:
        title: {type: string}
        artist: {type: string}
        genre: {type: string}
        duration_sec: {type: integer}
        year: {type: integer}
        rating: {type: number}
responses:
    201:
        description: Созданное произведение
    400:
        description: Ошибка валидации
"""
global next_id
data = request.get_json(silent=True) or {}
errors = validate_payload(data, partial=False)
if errors:
    return jsonify({"errors": errors}), 400
track = {"id": next_id, **{f: data[f] for f in ALL_FIELDS}}
tracks.append(track)
next_id += 1
return jsonify(track), 201

```

```

@app.route("/tracks/<int:track_id>", methods=["PUT"])

```

```

def update_track(track_id):

```

```

    """

```

```

    Обновить произведение по id

```

```

    ---

```

```

    parameters:

```

```

        - name: track_id

```

```

            in: path

```

```

            type: integer

```

```

            required: true

```

```

        - name: body

```

```

            in: body

```

```

            required: true

```

```

        schema:

```

```

            type: object

```

```

            properties:

```

```

                title: {type: string}

```

```

                artist: {type: string}

```

```

                genre: {type: string}

```

```

        duration_sec: {type: integer}
        year: {type: integer}
        rating: {type: number}
responses:
  200:
    description: Обновлённое произведение
  400:
    description: Ошибка валидации
  404:
    description: Произведение не найдено
"""
track = find_track(track_id)
if track is None:
    return jsonify({"error": "track not found"}), 404
data = request.get_json(silent=True) or {}
errors = validate_payload(data, partial=True)
if errors:
    return jsonify({"errors": errors}), 400
track.update({f: data[f] for f in ALL_FIELDS if f in data})
return jsonify(track)

@app.route("/tracks/<int:track_id>", methods=["DELETE"])
def delete_track(track_id):
    """
    Удалить произведение по id
    ---
    parameters:
      - name: track_id
        in: path
        type: integer
        required: true
    responses:
      204:
        description: Произведение удалено
      404:
        description: Произведение не найдено
    """
    track = find_track(track_id)
    if track is None:
        return jsonify({"error": "track not found"}), 404
    tracks.remove(track)
    return "", 204

```

```
if __name__ == "__main__":  
    app.run(debug=True, port=5001)
```